ECHOFY

Modern Music Streaming Web Application

Product Requirements Document (PRD)

Version: 1.0
Project Type: Full Stack Web Application
Project Duration: 3–4 Months
Development Methodology: Agile
Platform: Web Application

1. Project Overview

Project Name

Echofy

Project Description

Echofy is a modern music streaming web application that allows artists to upload and manage their music while listeners can stream songs, create playlists, save favorites, and discover trending music.

The application follows a modern client-server architecture where React is used for the frontend, PHP REST API for the backend, and MySQL for database management.

2. Problem Statement

Many music streaming platforms are overloaded with features and require subscriptions. Echofy aims to provide a lightweight, responsive, and user-friendly platform that enables artists to publish music and listeners to enjoy seamless music streaming.

3. Project Objectives

- Provide online music streaming
- Enable artists to upload songs
- Allow listeners to create playlists
- Help users discover trending music
- Build a responsive React application
- Implement secure authentication
- Provide a modern UI/UX

4. Target Users

Artist/Admin

Responsible for managing music and content.

Listener/User

Responsible for streaming music and managing playlists.

5. User Roles

Actor 1 — Artist/Admin

Responsibilities:

- Login
- Upload songs
- Upload cover images
- Create albums
- Edit songs
- Delete songs

- Manage artist profile
- Manage genres
- View play counts
- View song likes
- Mark trending songs

Actor 2 — Listener/User

Responsibilities:

- Register
- Login
- Stream songs
- Search songs
- Search artists
- Search albums
- Create playlists
- Add favorites
- View recently played
- View artist profile
- Edit profile
- Dark mode

6. Functional Requirements

Authentication

- User Registration
- User Login
- Logout
- Password Hashing
- JWT Authentication
- Protected Routes

Music Streaming

- Stream MP3 songs
- Play
- Pause
- Next
- Previous
- Shuffle
- Repeat
- Seek Bar
- Volume Control

Song Upload

- Upload MP3
- Upload Cover Image
- Edit Song
- Delete Song
- Upload Album

Search

Search by:

- Song
- Artist
- Album
- Genre

Playlist

- Create Playlist
- Rename Playlist
- Delete Playlist
- Add Song
- Remove Song

Favorites

- Like Song
- Remove Favorite
- View Favorite Songs

Recently Played

Store:

- Song
- Played Time

Trending Music

Display:

- Most Played Songs
- Latest Songs
- Popular Artists

Artist Profile

Display:

- Profile Image
- Biography
- Albums
- Songs
- Total Plays

User Profile

Edit:

- Name
- Email
- Password
- Profile Picture

Theme

- Light Mode
- Dark Mode

7. Non-Functional Requirements

- Responsive UI
- Mobile Friendly
- Fast Loading
- Secure Authentication
- Easy Navigation
- Scalable Database
- Modern User Interface

8. Technology Stack

Layer	Technology
Frontend	React 19, Vite, React Router DOM, Axios, Bootstrap 5, React Icons, Framer Motion
Backend	PHP 8 (REST API), Apache Server (WampServer)
Database	MySQL
Dev Tools	VS Code, Git, GitHub, Postman, phpMyAdmin, WampServer

9. System Architecture

```
React Frontend
       │
       │ Axios API Requests
       ▼
PHP REST API (WampServer)
       │
       ▼
MySQL Database
```

10. Application Modules

#	Module
1	Authentication
2	Home
3	Music Player
4	Search
5	Artist Dashboard
6	Song Management
7	Albums
8	Playlists
9	Favorites
10	Recently Played
11	Profile
12	Settings

11. React Pages

Public

- Landing Page
- Login
- Register
- Forgot Password

User

- Home
- Search
- Artist Details
- Album Details
- Playlist
- Favorites
- Recently Played
- Profile
- Settings
- Music Player

Artist/Admin

- Dashboard
- Upload Song
- Manage Songs
- Manage Albums

- Analytics
- Profile

12. React Components

Layout

- Navbar
- Sidebar
- Footer

Music

- Song Card
- Artist Card
- Album Card
- Playlist Card
- Music Player
- Mini Player

UI

- Search Bar
- Theme Toggle
- Loading Spinner
- Toast Notification
- Modal

13. Backend APIs

Authentication

Method	Endpoint
POST	/api/register
POST	/api/login
POST	/api/logout

Songs

Method	Endpoint
GET	/api/songs
GET	/api/song/{id}
POST	/api/song
PUT	/api/song/{id}

Method	Endpoint
DELETE	/api/song/{id}

Albums

Method	Endpoint
GET	/api/albums
POST	/api/albums
PUT	/api/albums/{id}
DELETE	/api/albums/{id}

Artists

Method	Endpoint
GET	/api/artists
GET	/api/artist/{id}

Favorites

Method	Endpoint
POST	/api/favorite
DELETE	/api/favorite/{id}
GET	/api/favorites

Playlists

Method	Endpoint
GET	/api/playlists
POST	/api/playlists
PUT	/api/playlists/{id}
DELETE	/api/playlists/{id}

Search

Method	Endpoint
GET	/api/search

Recent

Method	Endpoint
GET	/api/recent

14. Database Tables

- Users
- Artists
- Songs
- Albums
- Genres
- Playlists
- PlaylistSongs
- Favorites
- RecentlyPlayed

15. Database Relationships

- One Artist → Many Songs
- One Artist → Many Albums
- One Album → Many Songs
- One User → Many Playlists
- One Playlist → Many Songs
- One User → Many Favorites
- One User → Many Recently Played Songs

16. Folder Structure

```
Echofy/
```

frontend/

```
frontend/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   │   ├── layout/
│   │   ├── music/
│   │   └── common/
│   ├── pages/
│   │   ├── auth/
│   │   ├── user/
│   │   └── artist/
│   ├── services/
│   ├── context/
│   ├── hooks/
│   ├── utils/
│   ├── routes/
│   ├── App.jsx
│   └── main.jsx
```

backend/

```
backend/
├── api/
│   ├── auth/
│   ├── songs/
│   ├── albums/
│   ├── playlists/
│   ├── artists/
│   ├── favorites/
│   └── recent/
├── uploads/
│   ├── songs/
│   └── covers/
├── config/
├── database/
├── middleware/
├── helpers/
└── index.php
```

17. React Libraries

```
npm create vite@latest frontend -- --template react
npm install
npm install react-router-dom
npm install axios
npm install bootstrap
npm install react-icons
npm install framer-motion
npm install react-toastify
```

18. UI Theme

Element	Value
Primary Color	#1DB954 (Emerald Green)
Secondary Color	#121212 (Dark Black)
Accent	#FFFFFF
Typography	Poppins

Theme:

- Modern
- Minimal
- Spotify-inspired
- Dark Mode First

19. Future Enhancements

- Premium Subscription
- Offline Downloads

- AI Music Recommendation
- Lyrics Support
- Artist Verification
- Push Notifications
- Music Sharing
- Podcast Streaming

20. Expected Outcome

Echofy will be a modern full-stack music streaming platform built with React, PHP REST APIs, and MySQL, providing artists with a platform to upload music and listeners with an engaging streaming experience. The application will feature secure authentication, responsive design, playlists, favorites, search, trending music, and artist profiles while following current web development best practices.

This PRD provides a solid foundation for a college final-year project while remaining realistic to implement using React + PHP + MySQL + WampServer. It is structured so you can later add premium features and scale the application if desired.